WEEK-4
Introduction and Point to Point Communications in MPI

Niharika Gawade
240905284
Roll no:30
CSE-D

1. Implement a simple MPI program to find out pow (x, rank) for all the processes where 'x' is the integer constant and 'rank' is the rank of the process.

```
#include <stdio.h>
#include <mpi.h>
#include <math.h>

int main(int argc, char *argv[])
{
   int rank, size;
   int x = 2;
   double result;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   result = pow(x, rank);

   printf("Process %d: pow(%d, %d) = %.0f\n",
         rank, x, rank, result);

   MPI_Finalize();
   return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p1.c -o p1 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p1
Process 0: pow(2, 0) = 1
Process 2: pow(2, 2) = 4
Process 3: pow(2, 3) = 8
Process 4: pow(2, 4) = 16
Process 1: pow(2, 1) = 2
```

2. Implement a program in MPI to toggle the character of a given string indexed by the rank of the process. Hint: Suppose the string is HELLO and there are 5 processes, then process 0 toggle 'H' to 'h', process 1 toggle 'E' to 'e' and so on.

```c
#include <stdio.h>
#include <mpi.h>
#include <ctype.h>

int main(int argc, char *argv[])
{
   int rank, size;
   char str[] = "HELLO";

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (rank < 5)
   {
      str[rank] = tolower(str[rank]);

      printf("Process %d: %c\n", rank, str[rank]);
   }

   MPI_Finalize();
   return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p2.c -o p2 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p2
Process 0: h
Process 1: e
Process 2: l
Process 3: l
Process 4: o
```

3. Implement a program in MPI where even ranked process prints factorial of the rank and odd ranked process prints ranks Fibonacci number.

```c
#include <stdio.h>
#include <mpi.h>

int factorial(int n)
{
   int f = 1;
   for (int i = 1; i <= n; i++)
      f = f * i;

   return f;
}

int fibonacci(int n)
```

```
{
   int a = 0, b = 1, c;

   for (int i = 0; i < n; i++)
   {
      c = a + b;
      a = b;
      b = c;
   }

   return a;
}

int main(int argc, char *argv[])
{
   int rank;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank % 2 == 0)
      printf("Process %d: Factorial = %d\n",
            rank, factorial(rank));
   else
      printf("Process %d: Fibonacci = %d\n",
            rank, fibonacci(rank));

   MPI_Finalize();
   return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p3.c -o p3 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p3
Process 1: Fibonacci = 1
Process 3: Fibonacci = 2
Process 4: Factorial = 24
Process 0: Factorial = 1
Process 2: Factorial = 2
```

4. Implement a MPI program using synchronous send. The sender process sends a word to the receiver. The second process receives the word, toggles each letter of the word and sends it back to the first process. Both processes use synchronous send operations.

```
#include <stdio.h>

#include <string.h>
#include <ctype.h>
#include <mpi.h>

int main(int argc, char *argv[])
```

```
{
   int rank;
   char word[100];

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
   {
      strcpy(word, "HELLO");

      MPI_Ssend(word, strlen(word) + 1, MPI_CHAR, 1, 0,
            MPI_COMM_WORLD);

      MPI_Recv(word, 100, MPI_CHAR, 1, 1,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

      printf("Process 0 received: %s\n", word);
   }
   else if (rank == 1)
   {
      MPI_Recv(word, 100, MPI_CHAR, 0, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

      for (int i = 0; word[i] != '\0'; i++)
         word[i] = tolower(word[i]);

      MPI_Ssend(word, strlen(word) + 1, MPI_CHAR, 0, 1,
            MPI_COMM_WORLD);
   }

   MPI_Finalize();
   return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p4.c -o p4 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p4
Process 0 received: dessert
```

5. Implement a MPI program where the master process (process 0) sends a number to each of the slaves and the slave processes receive the number and prints it. Use standard send.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;
   int num = 10;
```

```
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0)
    {
        for (int i = 1; i < size; i++)
        {
            MPI_Send(&num, 1, MPI_INT, i, 0,
                    MPI_COMM_WORLD);
        }
    }
    else
    {
        MPI_Recv(&num, 1, MPI_INT, 0, 0,
                MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process %d received %d\n", rank, num);
    }

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p5.c -o p5 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p5
Process 1 received 10
Process 2 received 10
Process 3 received 10
Process 4 received 10
```

6. Implement a MPI program to read an integer value in the root process. Root process sends this value to Process1, Process 1 sends this value to Process2 and so on. Last process sends the value back to root process. When sending the value each process will first increment the received value by one. Implement the program using point to point communication routines.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int value;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    if (rank == 0)
    {
        printf("Enter a number: ");
        scanf("%d", &value);

        value++;

        MPI_Send(&value, 1, MPI_INT, 1, 0,
                MPI_COMM_WORLD);

        MPI_Recv(&value, 1, MPI_INT, size - 1, 0,
                MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Final value at root = %d\n", value);
    }
    else
    {
        MPI_Recv(&value, 1, MPI_INT, rank - 1, 0,
                MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        value++;

        if (rank == size - 1)
        {
            MPI_Send(&value, 1, MPI_INT, 0, 0,
                    MPI_COMM_WORLD);
        }
        else
        {
            MPI_Send(&value, 1, MPI_INT, rank + 1, 0,
                    MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p6.c -o p6 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p6
Enter a number: 4
Process 0: 5
Process 1: 6
Process 2: 7
Process 3: 8
Process 4: 9
Final value at root = 9
```

7. Implement a MPI program to read N elements of the array in the root process (process 0) where N is equal to the total number of processes. The root process sends one value to each of

the slaves. Let even ranked process finds square of the received element and odd ranked process finds cube of received element. Use Buffered send.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;
   int *arr;
   int value;
   int buffer[100];

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (rank == 0)
   {
      int a[100];

      printf("Enter %d elements:\n", size);

      for (int i = 0; i < size; i++)
         scanf("%d", &a[i]);

      MPI_Buffer_attach(buffer, sizeof(buffer));

      for (int i = 1; i < size; i++)
      {
         MPI_Bsend(&a[i], 1, MPI_INT, i, 0,
                  MPI_COMM_WORLD);
      }

      value = a[0];

      if (rank % 2 == 0)
         printf("Process 0: Square = %d\n", value * value);

      MPI_Buffer_detach(buffer, &(int){0});
   }
   else
   {
      MPI_Recv(&value, 1, MPI_INT, 0, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

      if (rank % 2 == 0)
         printf("Process %d: Square = %d\n",
               rank, value * value);
      else
         printf("Process %d: Cube = %d\n",
```

```
            rank, value * value * value);
    }

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpicc p7.c -o p7 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_4$ mpirun -np 5 ./p7
Enter 5 elements:
4
6
2
1
8
Process 0: Square = 16
Process 1: Cube = 216
Process 2: Square = 4
Process 3: Cube = 1
Process 4: Square = 64
```